

Final game_engine.py:

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.secret_number = random.randint(1, 100)

        self.low_bound = 1
        self.high_bound = 100

        self.attempts = 0
        self.max_attempts = 7
        self.guess_history = []

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)
        self.game_won = False
        self.game_over = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)
        self.font_history = pygame.font.SysFont(None, 24)

    def submit_guess(self):
        if self.game_won or self.game_over:
            return

        # Empty input does not consume an attempt.
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
```

```

        self.feedback_color = (240, 200, 80)
        return

guess = int(self.input_box.text)

# Every valid guess consumes one attempt.
self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
    self.low_bound = guess + 1

    self.guess_history.append((guess, "TOO LOW"))

    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)

elif guess > self.secret_number:
    self.high_bound = guess - 1

    self.guess_history.append((guess, "TOO HIGH"))

    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)

else:
    self.guess_history.append((guess, "CORRECT"))

    self.feedback_msg = (
        f"CORRECT! Found in {self.attempts} attempts."
    )
    self.feedback_color = (80, 220, 90)
    self.game_won = True

# Keep only the last 5 guesses.
self.guess_history = self.guess_history[-5:]

# If the player has used all 7 attempts without winning,
# end the game.
if self.attempts >= self.max_attempts and not self.game_won:

```

```

        self.game_over = True
        self.feedback_msg = (
            f"GAME OVER! The number was {self.secret_number}."
        )
        self.feedback_color = (240, 100, 80)

def reset(self):
    self.secret_number = random.randint(1, 100)

    self.low_bound = 1
    self.high_bound = 100

    self.attempts = 0
    self.guess_history = []

    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.game_over = False

    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_RETURN:
            self.submit_guess()

        elif event.key == pygame.K_r and (self.game_won or self.game_over):
            self.reset()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):

```

```
screen.fill((30, 34, 42))
```

```
title_surf = self.font_title.render(  
    "Number Guessing Arena",  
    True,  
    (245, 245, 245),  
)  
screen.blit(  
    title_surf,  
    (  
        self.width // 2 - title_surf.get_width() // 2,  
        35,  
    ),  
)
```

```
attempts_surf = self.font_medium.render(  
    f"Attempts: {self.attempts}/{self.max_attempts}",  
    True,  
    (180, 185, 195),  
)  
screen.blit(  
    attempts_surf,  
    (  
        self.width // 2 - attempts_surf.get_width() // 2,  
        95,  
    ),  
)
```

```
range_surf = self.font_medium.render(  
    f"Current Possible Range: {self.low_bound} - {self.high_bound}",  
    True,  
    (200, 210, 220),  
)  
screen.blit(  
    range_surf,  
    (  
        self.width // 2 - range_surf.get_width() // 2,  
        125,  
    ),  
)
```

```

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (50, 150, 80),
    self.submit_btn,
    border_radius=6,
)
pygame.draw.rect(
    screen,
    (220, 220, 220),
    self.submit_btn,
    width=2,
    border_radius=6,
)

btn_text = self.font_btn.render(
    "SUBMIT",
    True,
    (255, 255, 255),
)
screen.blit(
    btn_text,
    (
        self.submit_btn.centerx - btn_text.get_width() // 2,
        self.submit_btn.centery - btn_text.get_height() // 2,
    ),
)

feedback_surf = self.font_medium.render(
    self.feedback_msg,
    True,
    self.feedback_color,
)
screen.blit(
    feedback_surf,
    (
        self.width // 2 - feedback_surf.get_width() // 2,
        235,

```

```

    ),
)

# Guess history panel
history_title = self.font_medium.render(
    "Guess History",
    True,
    (245, 245, 245),
)
screen.blit(
    history_title,
    (
        self.width // 2 - history_title.get_width() // 2,
        285,
    ),
)

history_y = 320

for guess, result in reversed(self.guess_history):
    if result == "TOO LOW":
        history_text = f"{guess} \u2193 TOO LOW"
        history_color = (80, 160, 240)

    elif result == "TOO HIGH":
        history_text = f"{guess} \u2191 TOO HIGH"
        history_color = (240, 100, 80)

    else:
        history_text = f"{guess} \u2713 CORRECT"
        history_color = (80, 220, 90)

    history_surf = self.font_history.render(
        history_text,
        True,
        history_color,
    )
    screen.blit(
        history_surf,
        (

```

```

        self.width // 2 - history_surf.get_width() // 2,
        history_y,
    ),
)

history_y += 28

# Victory state
if self.game_won:
    restart_surf = self.font_medium.render(
        "Press [R] to Start a New Game",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            history_y + 15,
        ),
    )

# Game over state
elif self.game_over:
    game_over_surf = self.font_medium.render(
        "GAME OVER",
        True,
        (240, 100, 80),
    )
    screen.blit(
        game_over_surf,
        (
            self.width // 2 - game_over_surf.get_width() // 2,
            history_y + 15,
        ),
    )

secret_surf = self.font_medium.render(
    f"The secret number was {self.secret_number}",
    True,

```

```

        (245, 245, 245),
    )
    screen.blit(
        secret_surf,
        (
            self.width // 2 - secret_surf.get_width() // 2,
            history_y + 50,
        ),
    )

    restart_surf = self.font_medium.render(
        "Press [R] to Restart",
        True,
        (255, 220, 80),
    )
    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            history_y + 85,
        ),
    )

```

File name: `game_engine.py`

Final implementation includes: empty-input handling, dynamic range, last 5 guess history, 7-attempt limit, victory state, GAME OVER state, secret-number reveal, and restart with `R`.